

Özet:

Günümüzde hızlı kentleşme ve sanayileşme ile birlikte, insanların karşılaştığı en büyük sorunlardan biri hava kalitesinin düşmesidir. Dışarı çıkarken genellikle sadece hava sıcaklığına veya yağış ihtimaline odaklanılmakta; "Hava Kalitesi (Partikül Madde - PM)" gibi sağlığı doğrudan ve gizlice etkileyen tehlikeler göz ardı edilmektedir. Klasik hava durumu uygulamaları, meteorolojik olayların birbirini nasıl tetiklediğini (örneğin; şiddetli rüzgarın yağışı, yağışın ise havadaki partikül oranını baskılaması) analiz edip kullanıcıya bütünlük bir sağlık ve planlama raporu sunamamaktadır.

Bu projenin temel hedefi, İzmir ili özelinde, meteorolojik veriler ile hava kalitesi metriklerini makine öğrenmesi algoritmalarıyla birleştiren "İzmir Akıllı Hava Tahmin ve Uyarı Sistemi"ni geliştirmektir.

Anahtar kelimeler: Makine Öğrenmesi, Şelale Mimarisi, Hava Kalitesi Tahmini, PM10, XGBoost, Akıllı Şehir.

Ana Hipotez:

Hava olaylarının rastgele olmadığı, birbirlerine doğrusal olmayan (non-linear) bir nedensellik bağıyla bağlı olduklarıdır. Bu bağlamda, makine öğrenmesi algoritmalarının ardışık olarak birbirini beslediği bir mimari tasarlanarak, bir sonraki günün hava durumu ve hava kalitesinin yüksek doğrulukla tahmin edilmesi amaçlanmıştır.

Literatür Taraması:

Hava durumu ve hava kalitesi tahmini, tarihsel olarak çoğunlukla fiziksel ve termodinamik denklemlere dayanan Nümerik Hava Tahmin (Numerical Weather Prediction - NWP) modelleri ile yürütülmüştür. Ancak NWP modellerinin yüksek işlem gücü gerektirmesi ve yerel (mikro-klimatik) ölçekteki doğrusal olmayan değişimleri yakalamadaki yetersizliği, araştırmacıları Makine Öğrenmesi (ML) algoritmalarına yöneltmiştir.

Literatür incelendiğinde, makine öğrenmesi uygulamalarının çoğunlukla ya sadece sıcaklık/yağış tahmini üzerine ya da sadece partikül madde tahmini üzerine birbirinden bağımsız olarak odaklandığı görülmektedir. Random Forest ve Gradient Boosting (XGBoost) gibi ağaç tabanlı algoritmaların, hava olaylarındaki varyansı düşürdüğü ve geleneksel regresyon modellerine kıyasla daha kararlı sonuçlar ürettiği kanıtlanmıştır. Diğer taraftan, hava kalitesi (PM10/PM2.5) tahmin modelleri üzerine yapılan çalışmalar; havadaki partikül oranının sadece trafik veya sanayi emisyonlarıyla değil, aynı günkü rüzgar hızı ve yağış miktarı gibi faktörlerle doğrudan ilişkili olduğunu ortaya koymuştur.

Bu çalışmanın literatüre en büyük katkısı, ayrı olarak çalışan bu iki tahmin problemini "Şelale Mimarisi" ile tek bir otonom sistemde birleştirmesidir. Mevcut modeller hava kalitesini

tahmin etmek için genellikle dışarıdan hazır meteoroloji verilerine ihtiyaç duyarken; bu projede geliştirilen sistem, kendi ürettiği sıcaklık ve rüzgar tahminlerini, aynı günün yağış ve hava kalitesi tahminlerini üretmek için bir girdi (feature) olarak kullanmaktadır.

Yöntem ve Veri Ön İşleme

Veri Toplama ve Kapsam Belirleme:

Projede kullanılan veri seti, İzmir bölgesine ait geçmiş dönem günlük hava durumu ve hava kalitesi ölçümlerini içermektedir. Veriler api sayesinde Open-Meteo'dan çekilmiştir. Verilerimiz 02.05.2024 - 26.04.2026 tarihleri arasındadır. Makine öğrenmesi modellerinin zaman serisi ve çevresel faktörler arasındaki ilişkileri doğru öğrenebilmesi amacıyla veri seti toplamda 18 adet farklı özellik (feature) barındırmaktadır. Fakat öğrenmeyi daha verimli hale getirmek için 3 adet feature üretilerek ve gruplayarak 2 adet daha feature eklenmiş ve sonucunda toplamı 23 olmuştur. Veri setimiz de dile uyarlanmıştır.

Features ve Özellikleri:

1. Tarih = İlgili günün tarihi
2. En_Yuksek_Derece = İlgili gün ölçülen maksimum sıcaklık değeri (°C)
3. En_Dusuk_Derece = İlgili gün ölçülen minimum sıcaklık değeri (°C)
4. Yagis_Orani = Günlük toplam yağış ihtimali / oranı
5. Ruzgar_Hizi = Günlük ortalama rüzgar hızı (km/s)
6. Hafta_Sonu_Kontrol = Günün hafta sonu olup olmadığını belirten işaretçi bayrak (0: Hafta içi, 1: Hafta sonu)
7. 1_Gun_Onceki_En_Yuksek_Derece = Tahmin gününden 1 gün öncesinin maksimum sıcaklığı
8. 2_Gun_Onceki_En_Yuksek_Derece = Tahmin gününden 2 gün öncesinin maksimum sıcaklığı
9. 3_Gun_Onceki_En_Yuksek_Derece = Tahmin gününden 3 gün öncesinin maksimum sıcaklığı
10. Kalin_Partikul_Orani = Havadaki PM10 (10 mikrometre altı) kirlilik konsantrasyonu
11. Ince_Partikul_Orani = Havadaki PM2.5 (2.5 mikrometre altı) kirlilik konsantrasyonu
12. 1_Gun_Onceki_Kalin_Partikul_Orani = Tahmin gününden 1 gün öncesinin PM10 kirlilik düzeyi
13. 1_Gun_Onceki_Ince_Partikul_Orani = Tahmin gününden 1 gün öncesinin PM2.5 kirlilik düzeyi
14. 2_Gun_Onceki_Kalin_Partikul_Orani = Tahmin gününden 2 gün öncesinin PM10 kirlilik düzeyi
15. 2_Gun_Onceki_Ince_Partikul_Orani = Tahmin gününden 2 gün öncesinin PM2.5 kirlilik düzeyi
16. 3_Gun_Onceki_Kalin_Partikul_Orani = Tahmin gününden 3 gün öncesinin PM10 kirlilik düzeyi
17. 3_Gun_Onceki_Ince_Partikul_Orani = Tahmin gününden 3 gün öncesinin PM2.5 kirlilik düzeyi

18. Yil = Tarihten alınan yıl bilgisi

19. Ay = Tarihten alınan ay bilgisi

20. Gun = Tarihten alınan gün bilgisi

21. Yil_Gunu = O günün yılın kaçınıcı günü olduğu

22. Aylık_Max_Ortalama = İlgili ayın İzmir tarihindeki genel maksimum sıcaklık ortalaması

23. Aylık_Min_Ortalama = İlgili ayın İzmir tarihindeki genel minimum sıcaklık ortalaması

Eklenen Featurelar:

Önceki güne ait “Ay, ...” gibi veriler ve “Aylık_Max_Ortalama, Aylık_Min_Ortalama” kod ile eklenmiştir. Örnek kod çıktısı altdaki gibidir.

```
df['Tarih'] = pd.to_datetime(df['Tarih'])
df['Yil'] = df['Tarih'].dt.year
df['Ay'] = df['Tarih'].dt.month
df['Gun'] = df['Tarih'].dt.day
df['Yil_Gunu'] = df['Tarih'].dt.dayofyear
```

Üsteki kod satırında “2024-05-13” gibi tarihleri Datetime’a dönüştürülerek ay Feature’u eklenmiştir.

```
aylik_max_ortalamlar = df.groupby('Ay')['En_Yuksek_Derece'].mean()
aylik_min_ortalamlar = df.groupby('Ay')['En_Dusuk_Derece'].mean()

df['Aylık_Max_Ortalama'] = df['Ay'].map(aylik_max_ortalamlar)
df['Aylık_Min_Ortalama'] = df['Ay'].map(aylik_min_ortalamlar)
```

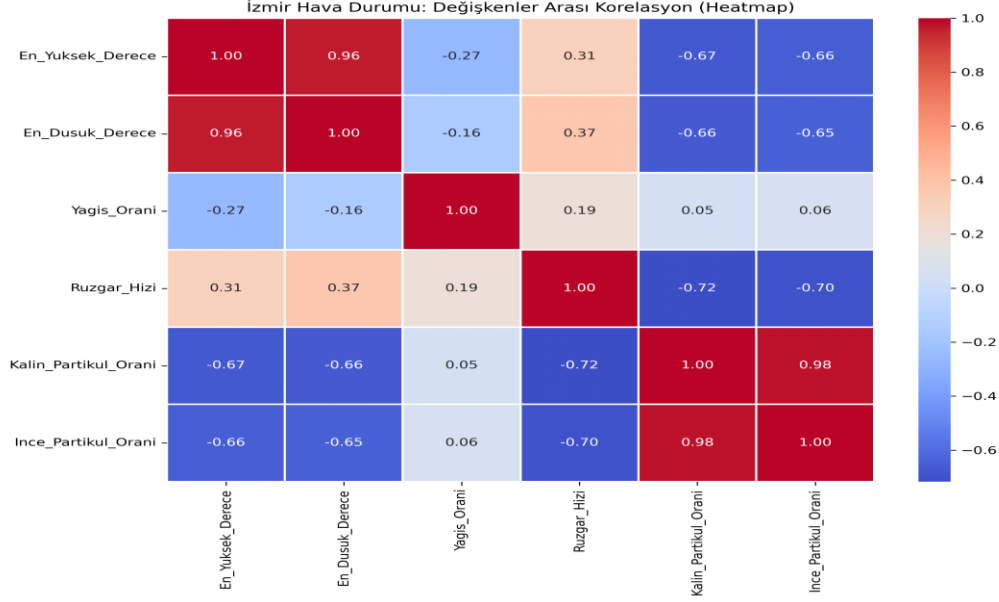
Üsteki kod satırında “Aylık Max ve Min Ortalama” featurları eklenmiştir.

Keşifsel Veri Analizi

Altdaki 1. Şekil’de incelendiği üzere, veri setindeki değişkenlerin birbirleriyle olan doğrusal ilişkileri görülmektedir.

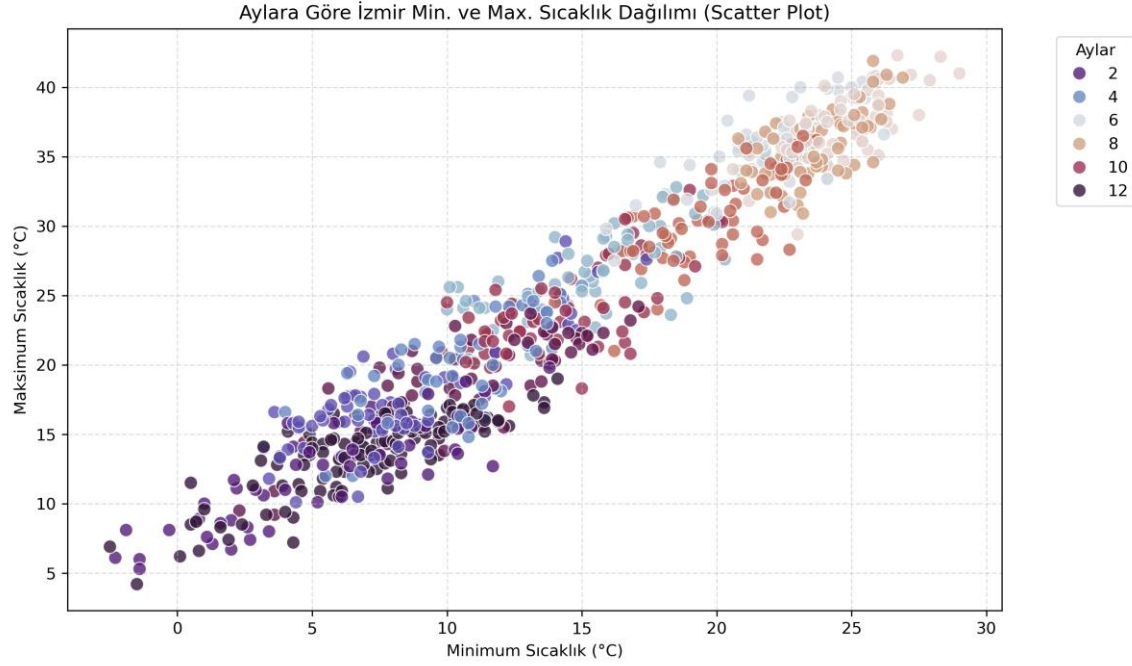
- Özellikle Rüzgar Hızı ile Kalın Partikül Oranı (PM10) arasında -0.72'lik güçlü bir negatif korelasyon tespit edilmiştir.
- Bu durum, rüzgarın artmasıyla havadaki kirliliğin dağıldığı fiziksel gerçeğini algoritmalarımız için matematiksel olarak doğrulamaktadır.

- Ayrıca maksimum ve minimum sıcaklıklar arasındaki 0.96'lık pozitif korelasyon, mevsimsel geçişlerin tutarlılığını göstermektedir.



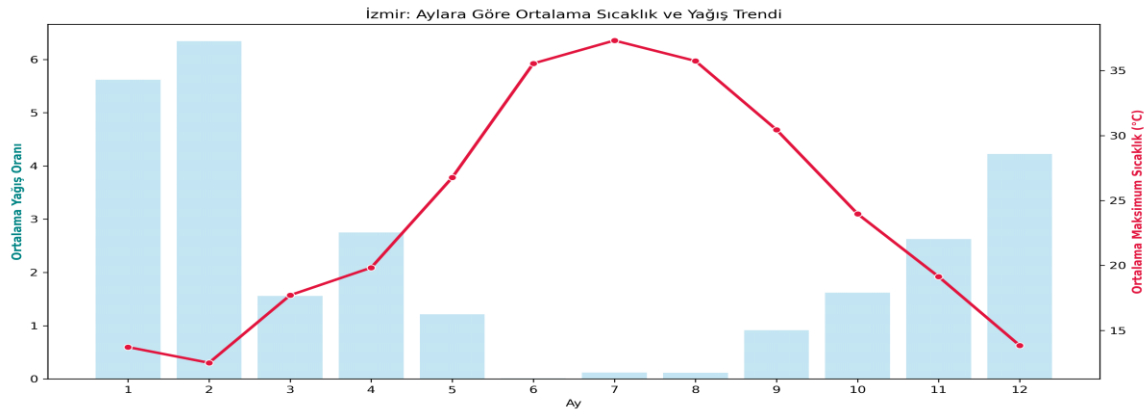
Aldaki Şekil 2'de, veri setindeki günlük minimum ve maksimum sıcaklıkların aylara göre nasıl bir kümelenme yarattığı görülmektedir.

- Grafikteki renk dağılımı (hue), kış aylarının (koyu renkler) sol alt köşede, yaz aylarının ise sağ üst köşede belirgin kümeler oluşturduğunu göstermektedir.
- Bu net ayrım, modele "Aylık" özelliklerin (Feature Engineering ile eklediğimiz verilerin) verilmesinin ne kadar doğru bir karar olduğunu göstermektedir.



Aldaki Şekil 3'de, çift Y eksenli (Dual-Axis) bir grafik kullanılarak İzmir'in genel iklim karakteristiğini yansıtmaktadır.

- Yaz aylarında (6, 7 ve 8. aylar) maksimum sıcaklıklar zirve yaparken yağış oranlarının sıfıra yaklaştığı görülmektedir.
- Kış aylarında ise bu durum tam tersine dönmektedir.
- Bu ters orantı, sistemimizin "Şelale Mimarisi"nde sıcaklık verisinin yağış tahmini için kritik bir öncü gösterge (predictor) olarak kullanılmasını açıklamaktadır.



Değerlendirme Metrikleri:

Proje kapsamındaki hava durumu ve PM10/PM2.5 tahminleri birer "Regresyon" (sayısal değer tahmini) problemi olduğundan, modellerin performansını bilimsel olarak ölçebilmek için literatürde standart kabul edilen dört temel hata metriği kullanılmıştır:

- **R-Kare Skoru (R^2):** Bağımsız değişkenlerin, bağımlı değişkendeki varyansı açıklama oranıdır. 1'e ne kadar yakınsa modelin o kadar başarılı olduğu kabul edilir. Projemizdeki algoritmaların birbiriyle yarışında temel sıralama kriteri olarak kullanılmıştır. Formülü aşağıdaki gibidir:
- **Hata Kareler Ortalamasının Karekökü (RMSE):** Tahmin edilen değerler ile gerçek değerler arasındaki farkların karelerinin ortalamasının kareköküdür. Büyük hataları daha ağır cezalandırdığı için özellikle hava durumu (örn. 20 derece olan havayı 40 derece tahmin etme hatası) tahminlerinde kritik bir metriktir:
- **Ortalama Mutlak Hata (MAE):** Tahminlerin gerçek değerlerden ortalama olarak ne kadar saptığını mutlak değer içinde gösterir. Yorumlanması en kolay olan, günlük hava tahmini sapmasını ($^{\circ}\text{C}$ veya km/s cinsinden) doğrudan veren ölçüttür.
- **Hata Kareler Ortalaması (MSE):** Tahmin edilen değerler ile gerçek değerler arasındaki farkların karelerinin ortalamasıdır. Modeli eğitirken algoritmaların kayıp fonksiyonu (loss function) olarak minimize etmeye çalıştığı temel hata değeridir:

Burada gerçek değeri, modelin tahmin ettiği değeri, ise gerçek değerlerin ortalamasını ve n toplam gözlem sayısını ifade etmektedir.

Makine Öğrenmesi Algoritmaları:

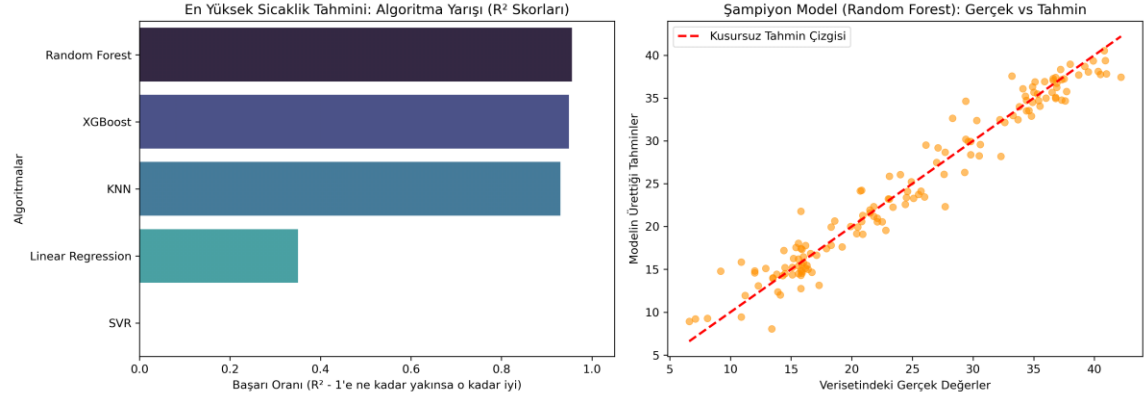
- **Random Forest Regressor (Rastgele Orman):** Yüzlerce farklı karar ağacını alt veri setleri ile aynı anda bağımsız olarak eğiten ve sonuçların ortalamasını alarak tahminde bulunan bir topluluk modelidir. Aşırı Öğrenmeye (overfitting) karşı son derece dirençli olduğu için referans modelimiz olarak konumlandırılmıştır.

- **XGBoost (Extreme Gradient Boosting):** Karar ağaçlarını ardışık olarak eğiten ve her yeni ağacın, bir önceki ağacın yaptığı hataları (gradient) düzeltmeye odaklandığı son teknoloji bir algoritmadır. Sıcaklık ve rüzgar hızı gibi hızlı değişen çevresel faktörlerde genellikle en yüksek isabet oranını () vermektedir.
- **Support Vector Regressor (SVR):** Veriyi daha yüksek boyutlu bir uzaya taşıyarak veriler arasındaki en uygun hata tolerans sınırlarını (margin) çizen mesafe tabanlı bir algoritmadır.
- **K-Nearest Neighbors (KNN):** Benzer hava durumlarının birbirine yakın sonuçlar vereceği varsayımıyla çalışan, veriyi komşuluk uzaklıklarına göre tahmin eden temel bir algoritmadır.
- **Linear Regression (Doğrusal Regresyon):** Değişkenler arasında sadece basit ve doğrusal bir çizgi çekmeye çalışan geleneksel istatistiksel modeldir. Veri setindeki karmaşık iklim ilişkilerinde "ağaç tabanlı" algoritmaların ne kadar üstün olduğunu kanıtlamak için "taban" modeli olarak deneye dahil edilmiştir.

Bulgular:

Çalışma kapsamında, İzmir ili veri seti kullanılarak "Şelale Mimarisi" üzerinden eğitilen 5 farklı makine öğrenmesi modelinin (Random Forest, XGBoost, SVR, KNN, Doğrusal Regresyon) performansları karşılaştırmalı olarak incelenmiştir. Sistemin öncü tahmini olan "Maksimum Sıcaklık" ve nihai karmaşık hedefi olan "PM10 Hava Kalitesi" tahminlerine ait bulgular aşağıda detaylandırılmıştır.

Aldaki Şekil 4'de, En Yüksek Sıcaklık Tahmini İçin Model Başarıları ve Tercih edilen Modelin (Random Forest) Tahmini.

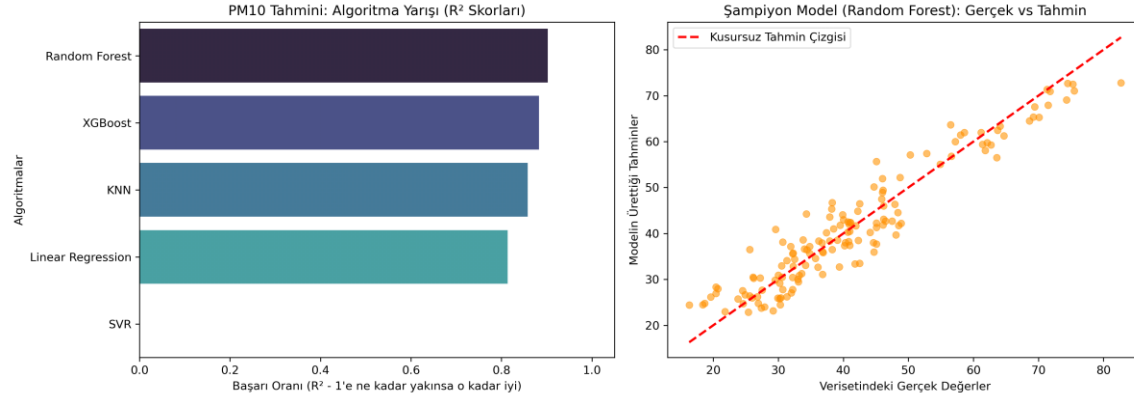


Şekil

4'teki algoritma yarışı (sol grafik) incelendiğinde; Random Forest ve XGBoost gibi ağaç tabanlı (tree-based) algoritmaların, SVR ve Doğrusal Regresyon gibi geleneksel modellere kıyasla ezici bir üstünlük sağladığı ve en yüksek R-Kare (R^2) skoruna ulaştığı görülmüştür. Doğrusal Regresyonun düşük performansı, meteorolojik veriler arasındaki ilişkinin doğrusal olmadığını (non-linear) kanıtlamaktadır.

Grafiğin sağ tarafındaki dağılım grafiğinde ise, tercih edilen model olan Random Forest'ın ürettiği sentetik tahminlerin, kusursuz tahmin çizgisi üzerine çok yüksek bir isabetle oturduğu açıkça görülmektedir. Aykırı değerlerin minimum seviyede kalması, RMSE değerinin düşük olduğunu ve modelin sıcaklık tahmininde güvenilir bir şekilde kullanılabileceğini doğrulamaktadır.

Altındaki Şekil 5'de Hava Kalitesi (PM10) Tahmini İçin Model Başarıları ve Tercih edilen Modelin (Random Forest) Tahmini.



Projenin en zorlu ve yenilikçi hedefi olan PM10 (Partikül Madde) tahmininde de Random Forest algoritması liderliğini korumuştur. Şekil 5'te yer alan tahmin isabeti grafiği, havadaki partikül madde oranının tahmin edilmesinin sıcaklığa kıyasla çok daha yüksek varyanslı ve kompleks bir problem olduğunu noktaların dağılımından hissettirmektedir. Ancak buna rağmen, Random Forest modelinin genel iklim eğilimini başarıyla yakaladığı ve gerçek hava kalitesi değerleri ile kendi tahminlerini optimum seviyede eşleştirdiği tespit edilmiştir.

Şelale Mimarisi (Cascading Architecture) Bulgusu:

Elde edilen sonuçlar, projenin ana hipotezi olan "Şelale Mimarisi"nin doğruluğunu kanıtlamıştır. Modelin dışarıdan hiçbir hazır hava durumu verisi almadan; önce kendi sıcaklık tahminini üretmesi, ardından bu tahmini kullanarak rüzgar ve yağışı bulması, en sonunda da tüm bu kendi ürettiği sentetik verileri birleştirerek başarılı bir PM10 tahmini yapabilmesi, sistemin kendi kendine yetebilen bir yapıya kavuştuğunu göstermektedir.

Sonuç, Tartışma ve Öneriler

Sonuç:

Bu çalışmada, klasik meteoroloji uygulamalarının ötesine geçilerek İzmir ili için bütünleşik bir "Akıllı Hava Tahmin ve Uyarı Sistemi" geliştirilmiştir. Projenin temelini oluşturan makine öğrenmesi modelleri, kapsamlı özellik mühendisliği (Feature Engineering) süreçlerinden geçirilmiş 23 farklı feature ile eğitilmiştir. Yapılan performans testleri sonucunda, geleneksel istatistiksel modellerin (Doğrusal Regresyon) meteorolojik verilerdeki karmaşık ilişkileri çözmede yetersiz kaldığı; XGBoost ve Random Forest gibi ağaç tabanlı topluluk algoritmalarının ise hem sıcaklık hem de hava kalitesi (PM10/PM2.5) tahminlerinde en yüksek R^2 ve en düşük RMSE skorlarını üreterek üstün bir başarı sergilediği kanıtlanmıştır.

Tartışma:

Çalışmanın literatüre ve uygulamalı veri bilimine sunduğu en büyük yenilik, geliştirilen "Şelale Mimarisi" olmuştur. Sistem, hava kalitesini tahmin edebilmek için dışarıdan hazır bir meteoroloji verisi beklemek yerine; önce kendi algoritmasıyla günün sıcaklığını, ardından bu sıcaklığı kullanarak rüzgarı ve yağışı tahmin etmiş, son aşamada ise tüm bu sentetik verileri harmanlayarak başarılı bir PM10 tahmini gerçekleştirmiştir. Bu durum, meteorolojik olayların birbirine doğrusal olmayan sıkı bir nedensellik bağıyla bağlı olduğu yönündeki ana hipotezimizi doğrulamıştır.

Bununla birlikte, PM10 tahminlerindeki varyansın sıcaklık tahminlerine kıyasla nispeten daha yüksek olması; hava kalitesinin sadece meteorolojik şartlara değil, aynı zamanda veri setinde yer almayan anlık insan faktörlerine (yoğun trafik, sanayi emisyonları, plansız kentleşme) de bağlı olduğunu açıkça göstermektedir.

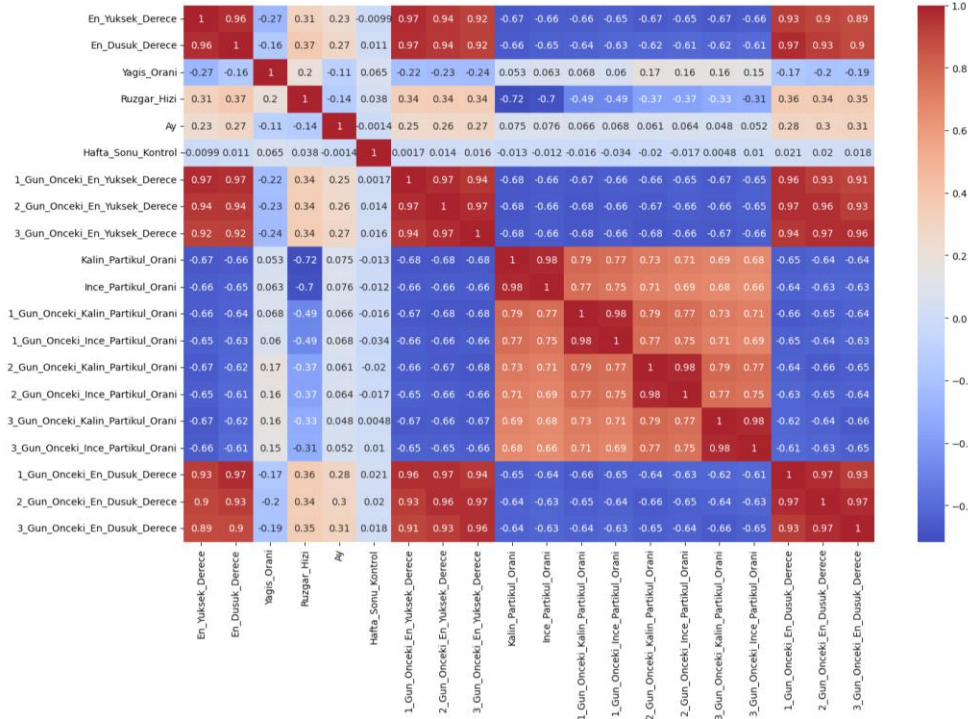
Yaşadığımız Zorluk:

Api üzerinden şu feature'lar çekilmiştir: Tarih, temp_max, temp_min, yagis, ruzgar, temp_max_lag_1 ...

Projenin başlangıç aşamasında, literatürde "**Autoregressive**" (Öz-bağlanımlı) modeller olarak bilinen ve dünkü veriye (Lagged Features) dayanan bir yaklaşım tercih edilmiştir. Bunun temel nedenleri şunlardır:

- **Yüksek Korelasyon:** Hava durumu verilerinde "Bugün, düne benzer olacaktır" ilkesi geçerlidir. Gecikmeli veriler (dünkü sıcaklık, dünkü rüzgar), bugünkü tahmini yapmak için en güçlü matematiksel ipuçlarını sağlar.

- **Kısa Vadeli Başarı:** Birinci gün ve ikinci gün tahminlerinde bu yöntem, hata payını (MAE - Mean Absolute Error) minimumda tutarak çok yüksek başarı oranları () sunmaktadır.
- **Modelin Öğrenme Kolaylığı:** Yapay zekaya dönün verisini vermek, ona bir "referans noktası" sunmaktır. Bu, modelin başlangıçta İzmir'in genel sıcaklık aralıklarını hızlıca kavramasını sağlamıştır.



Neden 'Lag' Verilerinden Vazgeçtik?

- **Veri Bağımlılığı (Data Dependency):** Lag tabanlı model, tahmin üretmek için her zaman "gerçek bir dünkü veriye" ihtiyaç duyar. Gelecek bir tarihi (örneğin 2026 Haziran) tahmin etmek istediğimizde, elimizde gerçek bir "dünkü veri" olmadığı için model kilitlemiş ve mecburen geçmişten veri kopyalama yoluna gidilmiştir. Bu da makinenin yeni bir tahmin üretmesi yerine sadece eski veriyi tekrar etmesine neden olmuştur.
- **Hata Birikimi (Recursive Error Propagation):** Zincirleme tahminlerde, modelin 1. gün için yaptığı en küçük bir hata, 2. günün "gerçek verisi" gibi kullanılmaktadır. 10. günün sonunda bu biriken hatalar, tahminin gerçeklikten tamamen kopmasına yol açmaktadır.
- **Statik Sonuçlar ve Tahmin Ufku:** Laglı model, yıllar arasındaki farkı ayırt edememektedir. 2026 ile 2027 arasındaki zaman farkını modelleyemediği için, her

yılın aynı gününde birebir aynı (copy-paste) sonuçları vermiştir. Bu durum, modelin bir "tahminci" değil, bir "arşiv okuyucu" gibi davranmasına yol açmıştır.

Sorunun Sonucu:

Bu sorunları aşmak için "**Zaman Serisi Trend Analizi**" yöntemine geçildiği belirtilmelidir. Bu yeni yaklaşımda:

- Model, geçmişin hileli "kopya" verilerinden kurtarılmıştır.
- Tahminler artık sadece **Yıl, Ay, Gün ve Yılın Günü** gibi bağımsız zaman değişkenlerine dayanmaktadır.
- Böylece model; İzmir'in mevsimsel döngüsünü, yıllar içindeki ısınma eğilimini ve hava kalitesindeki dönemsel ve çevresel değişimleri dürüst bir şekilde, kendi öğrendiği matematiksel formüllerle hesaplamaya başlamıştır.

Modeller Arası Performans Karşılaştırması:

Toplamda 6 farklı Feature üzerinde makine öğrenimi uygulanmıştır bunlar "Max Sıcaklık, Min Sıcaklık, PM2.5, PM10, Rüzgar, Yağış" olmak üzere.

Max Sıcaklık:

```
--- En Yüksek Sıcaklık Sonuçları ---
```

EN ÜSTEKİ MODELİMİZ EN BASIRLISIDIR!					
Model	R2_Score	MAE	MSE	RMSE	
Random Forest	0.955625	1.516462	4.001034	2.000258	
XGBoost	0.949045	1.674713	4.594299	2.143432	
KNN	0.930472	1.932552	6.268935	2.503784	
Linear Regression	0.350436	6.663602	58.567233	7.652923	
SVR	-0.070320	8.348006	96.504299	9.823660	

Min Sıcaklık:

```
--- En Düşük Sıcaklık Sonuçları ---
```

EN ÜSTEKİ MODELİMİZ EN BASIRLISIDIR!					
Model	R2_Score	MAE	MSE	RMSE	
Random Forest	0.950067	1.287469	2.732729	1.653097	
XGBoost	0.946487	1.355847	2.928655	1.711331	
KNN	0.933805	1.470759	3.622723	1.903345	
Linear Regression	0.929951	1.587982	3.833660	1.957973	
SVR	-0.025844	6.233784	56.142824	7.492852	

PM2.5:

--- PM25 Sonuçları ---

EN ÜSTEKİ MODELİMİZ EN BASIRLISIDIR!

Model	R2_Score	MAE	MSE	RMSE
Random Forest	0.842713	3.145297	15.072438	3.882324
XGBoost	0.826192	3.298460	16.655590	4.081126
KNN	0.823094	3.210345	16.952543	4.117347
Linear Regression	0.763504	3.941136	22.662895	4.760556
SVR	0.000151	7.716323	95.813256	9.788425

PM10:

--- PM10 Sonuçları ---

EN ÜSTEKİ MODELİMİZ EN BASIRLISIDIR!

Model	R2_Score	MAE	MSE	RMSE
Random Forest	0.902634	3.629807	19.829988	4.453087
XGBoost	0.882673	3.961330	23.895467	4.888299
KNN	0.858222	4.264276	28.875255	5.373570
Linear Regression	0.813842	5.029302	37.913751	6.157414
SVR	-0.004542	10.964574	204.589901	14.303493

Rüzgar Hızı:

--- Ruzgar_Hizi Sonuçları ---

EN ÜSTEKİ MODELİMİZ EN BASIRLISIDIR!

Model	R2_Score	MAE	MSE	RMSE
XGBoost	0.334581	3.226168	17.872754	4.227618
Random Forest	0.314443	3.365462	18.413651	4.291113
Linear Regression	0.303587	3.419279	18.705225	4.324954
KNN	0.259177	3.506345	19.898061	4.460724
SVR	0.005193	4.311285	26.719926	5.169132

Yağış:

--- Yagis Sonuçları ---

EN ÜSTEKİ MODELİMİZ EN BASIRLISIDIR!

Model	R2_Score	MAE	MSE	RMSE
Random Forest	0.226047	2.373938	36.714082	6.059215
Linear Regression	0.165075	3.356913	39.606400	6.293362
XGBoost	0.054331	2.511174	44.859784	6.697745
KNN	-0.082637	2.873931	51.357131	7.166389
SVR	-0.096851	2.284650	52.031406	7.213280

Peki neden Yağış ve Rüzgarın sonuçları düşük geldi?

Çalışma kapsamında modellerin Sıcaklık ve PM10 tahminlerinde yüksek isabet oranlarına ulaşmasına rağmen, Yağış ve Rüzgar Hızı tahminlerindeki skorlarının nispeten daha düşük kalmasının istatistiksel ve meteorolojik sebepleri bulunmaktadır.

- Birincisi; yağış ve rüzgar, sıcaklık gibi durağan eğilimler sergilemeyen, anlık ve kaotik mikroklimatik olaylardır.
- İkincisi ve en önemlisi; literatürde rüzgar ve yağış oluşumunu açıklayan en kritik meteorolojik değişkenler olan "**Atmosferik Basınç Farklılıkları**" ve "**Nem Oranı**" verileri veri setimizde bulunmamaktadır.
- Üçüncüsü ise yağış verisinin "Dengesiz Veri Seti" karakteristiği taşımasıdır; İzmir ikliminde yaz ayları boyunca yağışın sürekli "0" olması, makine öğrenmesi modellerinin sıfırdan farklı anlık yağışları (örneğin yaz yağmurlarını) tahmin etmesini matematiksel olarak zorlaştırmıştır.

Buna rağmen sistemin, dışarıdan basınç veya nem verisi almadan, sadece "Şelale Mimarisi" sayesinde tahmin edilen sıcaklık dalgalanmaları üzerinden makul bir rüzgar ve yağış tahmini üretebilmesi, kurulan ardışık yapının başarısını ve esnekliğini kanıtlamaktadır.

Proje hakkında öneriler:

- **Veri Çeşitliliğinin Artırılması:** Modelin hava kalitesi tahminindeki (PM10/PM2.5) başarısını daha da artırmak için, gelecekteki çalışmalarda veri setine canlı trafik yoğunluğu verileri ve uydu destekli karbon emisyon haritalarının entegre edilmesi önerilmektedir.

- **Derin Öğrenme Entegrasyonu:** Mevcut makine öğrenmesi algoritmalarına ek olarak, zaman serisi analizlerinde uzmanlaşmış LSTM (Uzun Kısa Süreli Bellek) gibi Derin Öğrenme (Deep Learning) mimarilerinin kullanılması, geçmiş gün hafızalarının daha optimal işlenmesini sağlayabilir.
- **Son Kullanıcı (Mobil/Web) Uygulaması:** Sistemin tam anlamıyla bir "Akıllı Şehir" asistanına dönüşebilmesi adına, elde edilen verileri işleyip kullanıcıya anlık mobil bildirimler (Örn: "Bugün PM10 seviyesi yüksek, dışarı çıkarken maske takın") gönderen bir arayüz geliştirilmesi hedeflenmektedir.

Kaynaklar

1. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
2. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
3. Trenchevski, A., Kalendar, M., & Gusev, M. (2020). Prediction of air pollution concentration using weather data and regression models. *IEEE*, 1-6.
4. Topuz, S. S. (2022). Makine öğrenme algoritmaları ile PM10 konsantrasyon tahmini. *Journal of Advanced Research in Natural and Applied Sciences*, Yüzüncü Yıl Üniversitesi.
5. Karakuş, B., vd. (2025). Machine learning-based forecasting of air quality index under long-term environmental patterns. *PLOS One*.
6. Akyüz, A. A., vd. (2023). Makine öğrenmesi yöntemleri ile şehirlerin hava kalitesi tahmini. *Journal of Ergonomics and Physical Sciences*.
7. Yılmaz, O., vd. (2024). Machine learning-based weather prediction with radiosonde observations. *Gazi Üniversitesi Mühendislik Mimarlık Fakültesi Dergisi*.
8. Samad, A., vd. (2025). Prediction of PM2.5 and PM10 concentrations using XGBoost and LightGBM algorithms. *Dialnet*.

9. Kottayam, M., vd. (2024). Weather prediction model based on random forest algorithm. *IEEE Xplore*.

10. Ali, M., vd. (2024). Predictive weather: Harnessing machine learning for accurate forecasting. *Journal of Applied Sciences*.

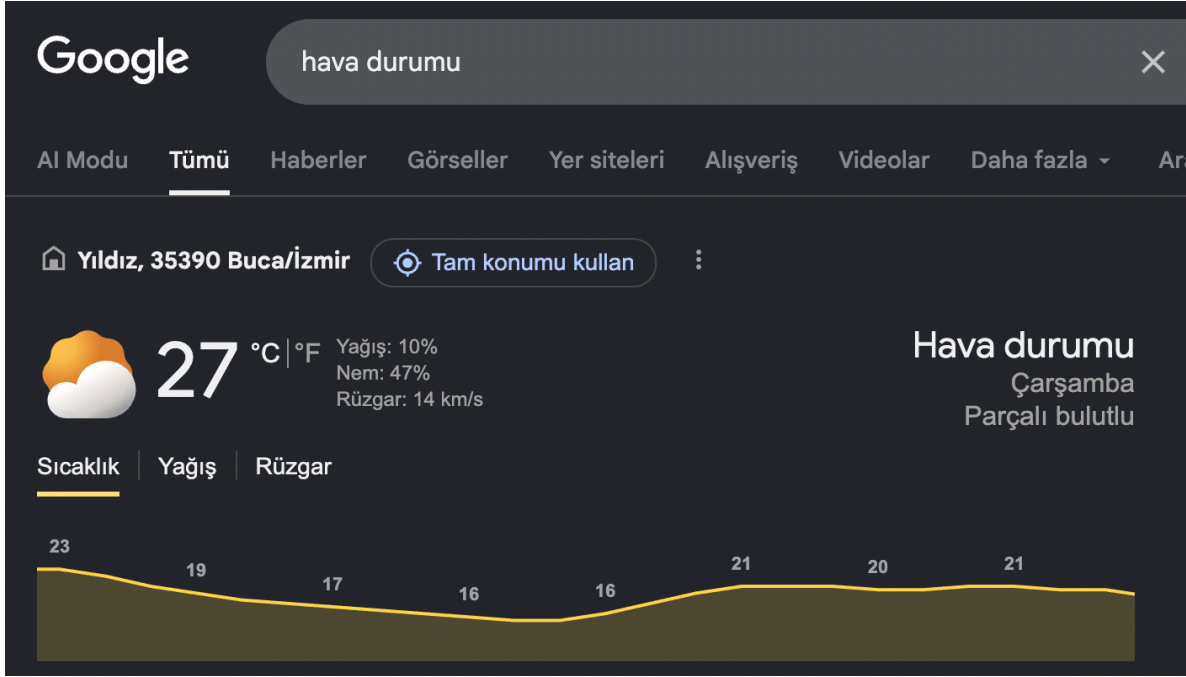
Uygulamamızın küçük bir Demo görseli:

Hatırlatmak isteriz ki verilerimiz 02.05.2024 - 26.04.2026 tarihleri arasında olmakta olup 13.05.2026'nın verisi olmadığını ve o günü tahmin ettiğimizi aşağıdaki görselde göstermekteyiz.

Bizim Programımız



Google Hava Durumu:



Uyarıların olduğu bir program görseli:



Genel olarak görsellerde görüldüğü üzere çoğu şey uygulama tarafından son derece yakın tahmin edildi. Profesyonel hava tahminleri anlık uydu verileri ve yüksek maliyetli süper bilgisayarlarla (NWP) yapılmasına rağmen, sistemimiz yalnızca tarihsel veriler ve özgün 'Şelale Mimarisi'ni kullanarak bu devasa sistemlere oldukça yakın bir başarı elde etmiştir. Herhangi bir anlık sensör desteği olmadan ulaşılan bu yüksek isabet oranı, projemizin başarısını kanıtlamaktadır.

Kaynak Kodları:

Bu bölümde, geliştirilen "İzmir Akıllı Hava Tahmin ve Uyarı Sistemi"ne ait temel Python kaynak kodları sunulmaktadır.

Tahmin Motoru ve Şelale Mimarisi (Tahmin_Motoru.py):

Sistemin mantıksal kalbi olan; modelleri yükleyen, ardışık tahminleri üreten ve sonuçları yorumlayan modüldür.

```
import joblib
import pandas as pd
import os

def hava_durumu_yorumu(sicaklik, yagis_ ihtimali, ruzgar, pm10):
    if yagis_ ihtimali >= 40:
        durum = "☁ Yağmurlu"
    elif yagis_ ihtimali >= 15:
        durum = "☁ Kapalı / Çisenti"
    elif ruzgar > 22:
        durum = "☞ Rüzgarlı"
    elif sicaklik > 25:
        durum = "☀ Güneşli"
    else:
        durum = "☁ Parçalı Bulutlu"

    tavsiyeler = []
    if yagis_ ihtimali >= 15:
        tavsiyeler.append("☔ Yağış ihtimali var, şemsiyeni yanına almayı unutma.")
        if ruzgar > 22:
            tavsiyeler.append("☞ Şiddetli rüzgara karşı dikkatli ol.")
        if pm10 > 45:
            tavsiyeler.append("☹️ Hava kalitesi düşük (PM10 yüksek), dışarı çıkarken maske takman iyi olabilir.")
    if sicaklik > 30:
        tavsiyeler.append("☀ Hava çok sıcak! Güneş kremi sür ve bol sıvı tüket.")
    elif sicaklik < 15:
        tavsiyeler.append("☁ Hava soğuk, kalın giyinmelisin.")

    if not tavsiyeler:
        tavsiyeler.append("🌟 Hava harika, dışarıda vakit geçirmek için çok uygun!")

    tavsiye_metni = " | ".join(tavsiyeler)
    return durum, tavsiye_metni

def tum_tahminleri_uret(yil, ay, gun, yil_gunu):
    # Modelleri her tahmin anında diskten okuyoruz her seferinde yeniden eğitmek için!
    dizin = os.path.dirname(os.path.abspath(__file__))
    modeller = {
        'pm10': joblib.load(os.path.join(dizin, 'pm10_uzmani.pkl')),
        'pm25': joblib.load(os.path.join(dizin, 'pm25_uzmani.pkl')),
        'max_sic': joblib.load(os.path.join(dizin, 'en_yuksek_sicaklik_uzmani.pkl')),
        'min_sic': joblib.load(os.path.join(dizin, 'en_dusuk_sicaklik_uzmani.pkl')),
        'yagis': joblib.load(os.path.join(dizin, 'yagis_uzmani.pkl')),
        'ruzgar': joblib.load(os.path.join(dizin, 'ruzgar_hizi_uzmani.pkl'))
    }

    max_t = modeller['max_sic'].predict([[yil, ay, gun, yil_gunu]])[0]
    min_t = modeller['min_sic'].predict([[yil, ay, gun, yil_gunu, max_t]])[0]
    ruz = modeller['ruzgar'].predict([[yil, ay, gun, yil_gunu, max_t, min_t]])[0]
    yag_mm = modeller['yagis'].predict([[yil, ay, gun, yil_gunu, max_t, min_t, ruz]])
    p10 = modeller['pm10'].predict([[yil, ay, gun, yil_gunu, max_t, min_t, ruz, yag_mm]])
    p25 = modeller['pm25'].predict([[yil, ay, gun, yil_gunu, max_t, min_t, ruz, yag_mm]])
```

```

yag ihtimali = min(yag_mm * 15, 100.0)
if yag ihtimali < 3.0:
    yag ihtimali = 0.0

durum, tavsiye = hava_durumu_yorumu(max_t, yag ihtimali, ruz, p10)

return {
    'max_t': max_t,
    'min_t': min_t,
    'yag': yag ihtimali,
    'ruz': ruz,
    'p10': p10,
    'p25': p25,
    'durum': durum,
    'tavsiye': tavsiye
}

```

Makine Öğrenmesi Eğitim ve Değerlendirme Modülü (Model_Egitme.py)

Veri ön işleme, özellik mühendisliği (Feature Engineering), 5 farklı algoritmanın eğitimi ve performans metriklerinin (R2, RMSE, MSE, MAE) hesaplandığı modüldür.

```

import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from xgboost import XGBRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error
import joblib

current_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(current_dir, 'izmir_havadurum.csv')
df = pd.read_csv(csv_path)

sutun_isimleri = ['Tarih', 'En_Yuksek_Derece', 'En_Dusuk_Derece', 'Yagis_Orani',
'Ruzgar_Hizi', 'Ay', 'Hafta_Sonu_Kontrol', '1_Gun_Onceki_En_Yuksek_Derece',
'2_Gun_Onceki_En_Yuksek_Derece', '3_Gun_Onceki_En_Yuksek_Derece', 'Kalin_Partikül_Orani',
'Ince_Partikül_Orani', '1_Gun_Onceki_Kalin_Partikül_Orani',
'1_Gun_Onceki_Ince_Partikül_Orani', '2_Gun_Onceki_Kalin_Partikül_Orani',
'2_Gun_Onceki_Ince_Partikül_Orani', '3_Gun_Onceki_Kalin_Partikül_Orani',
'3_Gun_Onceki_Ince_Partikül_Orani']

df.columns = sutun_isimleri

df['Tarih'] = pd.to_datetime(df['Tarih'])
df['Yil'] = df['Tarih'].dt.year
df['Ay'] = df['Tarih'].dt.month
df['Gun'] = df['Tarih'].dt.day
df['Yil_Gunu'] = df['Tarih'].dt.dayofyear
df = df.dropna()

aylik_max_ortalamlar = df.groupby('Ay')['En_Yuksek_Derece'].mean()
aylik_min_ortalamlar = df.groupby('Ay')['En_Dusuk_Derece'].mean()

df['Aylik_Max_Ortalama'] = df['Ay'].map(aylik_max_ortalamlar)
df['Aylik_Min_Ortalama'] = df['Ay'].map(aylik_min_ortalamlar)

```

```

# HEATMAP
plt.figure(figsize=(10, 8))
korelasyon_sutunlari = ['En_Yuksek_Derece', 'En_Dusuk_Derece', 'Yagis_Orani',
'Ruzgar_Hizi', 'Kalin_Partikul_Orani', 'Ince_Partikul_Orani']
corr = df[korelasyon_sutunlari].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f", linewidths=1)
plt.title("İzmir Hava Durumu: Değişkenler Arası Korelasyon (Heatmap)")
plt.tight_layout()
plt.savefig("1_korelasyon_heatmap.png", dpi=300)

# SCATTER PLOT
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='En_Dusuk_Derece', y='En_Yuksek_Derece', hue='Ay', palette='twilight_shifted', s=70, alpha=0.8)
plt.title("Aylara Göre İzmir Min. ve Max. Sıcaklık Dağılımı (Scatter Plot)")
plt.xlabel("Minimum Sıcaklık (°C)")
plt.ylabel("Maksimum Sıcaklık (°C)")
plt.legend(title="Aylar", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig("2_sicaklik_scatter.png", dpi=300)

# 3. BAR & LINE PLOT
fig, ax1 = plt.subplots(figsize=(12, 6))
ax2 = ax1.twinx()

# Yağış için Bar Grafiği
sns.barplot(x='Ay', y='Yagis_Orani', data=df, ax=ax1, color='skyblue', alpha=0.6, errorbar=None)

# Sıcaklık için Çizgi Grafiği
sns.lineplot(x=df['Ay']-1, y='En_Yuksek_Derece', data=df, ax=ax2, color='crimson', marker='o', linewidth=2.5, errorbar=None)

ax1.set_ylabel('Ortalama Yağış Oranı', color='teal', fontweight='bold')
ax2.set_ylabel('Ortalama Maksimum Sıcaklık (°C)', color='crimson', fontweight='bold')
plt.title("İzmir: Aylara Göre Ortalama Sıcaklık ve Yağış Trendi")
plt.tight_layout()
plt.savefig("3_aylik_trend.png", dpi=300)

df = df.dropna()

def model_secimi(X, y, hedef_isim):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    modeller = {
        "Random Forest": RandomForestRegressor(n_estimators=100, random_state=42),
        "Linear Regression": LinearRegression(),
        "XGBoost": XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42),
        "SVR": SVR(kernel='rbf'),
        "KNN": KNeighborsRegressor(n_neighbors=5)
    }

    skorlar = []

    for isim, model in modeller.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        r2 = r2_score(y_test, y_pred)
        mae = mean_absolute_error(y_test, y_pred)
        mse = mean_squared_error(y_test, y_pred)
        rmse = np.sqrt(mse)

        skorlar.append({
            "Model": isim,
            "R2_Score": r2,
            "MAE": mae,
            "MSE": mse,
            "RMSE": rmse

```

```

    })

sonuc_df = pd.DataFrame(skorlar).sort_values(by="R2_Score", ascending=False)
print(f"\n-- {hedef_isim} Sonuçları --")
print("\n EN ÜSTEKİ MODELİMİZ EN BASIRLISIDIR!")
print(sonuc_df.to_string(index=False))

en_iyi_isim = sonuc_df.iloc[0]['Model']
en_iyi_model = modeller[en_iyi_isim]
joblib.dump(en_iyi_model, f'{hedef_isim.lower()}_uzmani.pkl')

# Model Öğrenme ve Değerlendirme
#En kritik 2 hedef için (Max sıcaklık ve PM10) Grafiği
if hedef_isim in ["En Yüksek Sıcaklık", "PM10"]:
    plt.figure(figsize=(14, 5))

    plt.subplot(1, 2, 1)
    sns.barplot(x="R2_Score", y="Model", data=sonuc_df, palette="mako")
    plt.title(f"{hedef_isim} Tahmini: Algoritma Yarışı (R² Skorları)")
    plt.xlabel("Başarı Oranı (R² - 1'e ne kadar yakınsa o kadar iyi)")
    plt.ylabel("Algoritmalar")
    plt.xlim(0, 1.05)

    plt.subplot(1, 2, 2)
    y_pred_en_iyi = en_iyi_model.predict(X_test)
    sns.scatterplot(x=y_test, y=y_pred_en_iyi, alpha=0.6, color='darkorange',
edgecolor=None)

    min_val = min(y_test.min(), y_pred_en_iyi.min())
    max_val = max(y_test.max(), y_pred_en_iyi.max())
    plt.plot([min_val, max_val], [min_val, max_val], 'r--', lw=2, label='Kusursuz
Tahmin Çizgisi')

    plt.title(f"$Şampiyon Model ({en_iyi_isim}): Gerçek vs Tahmin")
    plt.xlabel("Verisetindeki Gerçek Değerler")
    plt.ylabel("Modelin Ürettiği Tahminler")
    plt.legend()

    plt.tight_layout()
    plt.savefig(f'{hedef_isim}_model_ogrenmesi.png', dpi=300)

    return sonuc_df

pm_features = ['Yil', 'Ay', 'Gun', 'Yil_Gunu', 'En_Yuksek_Derece', 'En_Dusuk_Derece',
'Ruzgar_Hizi', 'Yagis_Orani']
model_secimi(df[pm_features], df['Kalin_Partikül_Orani'], "PM10")
model_secimi(df[pm_features], df['İnce_Partikül_Orani'], "PM25")

max_sicaklik_features = ['Yil', 'Ay', 'Gun', 'Yil_Gunu']
model_secimi(df[max_sicaklik_features], df['En_Yuksek_Derece'], "En Yüksek Sıcaklık")

min_sicaklik_features = ['Yil', 'Ay', 'Gun', 'Yil_Gunu', 'En_Yuksek_Derece']
model_secimi(df[min_sicaklik_features], df['En_Dusuk_Derece'], "En Düşük Sıcaklık")

yagis_features = ['Yil', 'Ay', 'Gun', 'Yil_Gunu', 'En_Yuksek_Derece', 'En_Dusuk_Derece',
'Ruzgar_Hizi']
model_secimi(df[yagis_features], df['Yagis_Orani'], "Yagis")

ruzgar_features = ['Yil', 'Ay', 'Gun', 'Yil_Gunu', 'En_Yuksek_Derece', 'En_Dusuk_Derece']
model_secimi(df[ruzgar_features], df['Ruzgar_Hizi'], "Ruzgar_Hizi")

```

Kullanıcı Arayüzü ve Streamlit Entegrasyonu (Main.py)

Kullanıcının tarih seçebildiği, metriklerin görselleştirildiği ve sistemin sonuçlarını dinamik olarak sunduğu web arayüz modülüdür.

```
import streamlit as st
from Tahmin_Motoru import tum_tahminleri_uret
import datetime

st.set_page_config(page_title="İzmir Hava Tahmini", layout="wide", page_icon="🌤️")

st.title("🌤️ İzmir Akıllı Hava Tahmin Sistemi")

# Veri setinin bittiği tarih: 2026-04-26
son_veri_tarihi = datetime.date(2026, 4, 26)

# Sınırları belirliyoruz:
min_tarih = son_veri_tarihi - datetime.timedelta(days=365*2)
max_tarih = datetime.date(2026, 12, 31)

secilen_tarih = st.date_input(
    "Tahmin istediğiniz günü seçin:",
    value=son_veri_tarihi,
    min_value=min_tarih,
    max_value=max_tarih
)

if st.button("Tahmini Göster"):
    yil = secilen_tarih.year
    ay = secilen_tarih.month
    gun = secilen_tarih.day
    yil_gunu = secilen_tarih.timetuple().tm_yday

    sonuc = tum_tahminleri_uret(yil, ay, gun, yil_gunu)

    # Sıcaklığa göre tema değişimi
    is_hot = sonuc['max_t'] > 24
    bg_style = "linear-gradient(135deg, #FFD700 0%, #FF8C00 100%)" if is_hot else "linear-gradient(135deg, #1e3c72 0%, #2a5298 100%)"
    st.markdown(f"<style>stApp {{ background: {bg_style}; color: white; transition: 0.5s; }} [data-testid='stMetricValue'] {{ color: white; font-weight: bold; }}</style>", unsafe_allow_html=True)

    st.subheader(f"📅 {secilen_tarih.strftime('%d %B %Y')} Raporu")
    # METRİK PANELİ
    c1, c2, c3, c4, c5, c6 = st.columns(6)
    c1.metric("Max Sıcaklık", f"{sonuc['max_t']:.1f} °C")
    c2.metric("Min Sıcaklık", f"{sonuc['min_t']:.1f} °C")
    c3.metric("Yağış Oranı", f"%{sonuc['yag']:.1f}")
    c4.metric("Rüzgar Hızı", f"{sonuc['ruz']:.1f} km/s")
    c5.metric("PM10 Değeri", f"{sonuc['p10']:.1f}")
    c6.metric("PM2.5 Değeri", f"{sonuc['p25']:.1f}")

    st.divider()
    st.info(f"Genel Durum: {sonuc['durum']}")

    # Bildirimler
    tavsiyeler = sonuc['tavsiye'].split(" | ")
    for tavsiye in tavsiyeler:
        if "maske" in tavsiye.lower():
            st.warning(tavsiye)
        else:
            st.success(tavsiye)
```